

```

    m = x+(y-x)/2;
    if (A[m]>=v) y=m;
    else x=m+1;
}
return x;
}

```

下面来分析一下这段程序。首先，最后的返回值不仅可能是 x , $x+1$, $x+2, \dots$, $y-1$ ，还可能是 y ——如果 v 大于 $A[y-1]$ ，就只能插入这里了。这样，尽管查找区间是左闭右开区间 $[x, y)$ ，返回值的候选区间却是闭区间 $[x, y]$ 。 $A[m]$ 和 v 的各种关系所带来的影响如下。

- $A[m] = v$: 至少已经找到一个，而左边可能还有，因此区间变为 $[x, m]$ 。
- $A[m] > v$: 所求位置不可能在后面，但有可能是 m ，因此区间变为 $[x, m]$ 。
- $A[m] < v$: m 和前面都不可行，因此区间变为 $[m+1, y]$ 。

合并一下， $A[m] \geq v$ 时新区间为 $[x, m]$ ； $A[m] < v$ 时新区间为 $[m+1, y]$ 。这里有一个潜在的危险：如果 $[x, m]$ 或者 $[m+1, y]$ 和原区间 $[x, y]$ 相同，将发生死循环！幸运的是，这样的情况并不会发生，原因留给读者思考。

类似地，可以写一个 `upper_bound` 程序，当 v 存在时返回它出现的最后一个位置的后面一个位置。如果不存在，返回这样一个下标 i ：在此处插入 v （原来的元素 $A[i]$, $A[i+1]$, $\dots$ 全部往后移动一个位置）后序列仍然有序。不难得出，只需把 “`if(A[m]>=v) y=m; else x=m+1;`” 改成 “`if(A[m]<=v) x=m+1; else y=m;`” 即可。

这样，对二分查找的讨论就相对比较完整了：设 `lower_bound` 和 `upper_bound` 的返回值分别为 L 和 R ，则 v 出现的子序列为 $[L, R)$ 。这个结论当 v 不在时也成立：此时 $L=R$ ，区间为空。这里实现的 `lower_bound` 和 `upper_bound` 就是 STL 中的同名函数。

提示 8-11: 用“上下界”函数求解范围统计问题的技巧非常有用，建议读者用心体会左闭右开区间的使用方法和上下界函数的实现细节。

8.3 递归与分治

除了排序与检索外，递归还有更广泛的应用。

棋盘覆盖问题。有一个 $2^k \times 2^k$ 的方格棋盘，恰有一个方格是黑色的，其他为白色。你的任务是用包含 3 个方格的 L 型牌覆盖所有白色方格。黑色方格不能被覆盖，且任意一个白色方格不能同时被两个或更多牌覆盖。如图 8-3 所示为 L 型牌的 4 种旋转方式。

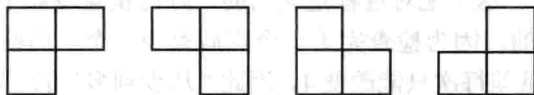


图 8-3 L 型牌

【分析】

本题的棋盘是 $2^k \times 2^k$ 的，很容易想到分治：把棋盘切为 4 块，则每一块都是 $2^{k-1} \times 2^{k-1}$ 的。有黑格的那一块可以递归解决，但其他 3 块并没有黑格子，应该怎么办呢？可以构造出一个黑格子，如图 8-4 所示。递归边界也不难得出： $k=1$ 时一块牌就够了。

循环日程表问题。 $n=2^k$ 个运动员进行网球循环赛，需要设计比赛日程表。每个选手必须与其他 $n-1$ 个选手各赛一次；每个选手一天只能赛一次；循环赛一共进行 $n-1$ 天。按此要求设计一张比赛日程表，该表有 n 行和 $n-1$ 列，第 i 行 j 列为第 i 个选手第 j 天遇到的选手。

【分析】

本题的方法有很多，递归是其中一种比较容易理解的方法。如图 8-5 所示是 $k=3$ 时的一个可行解，它是 4 块拼起来的。左上角是 $k=2$ 时的一组解，左下角是左上角每个数加 4 得到，而右上角、右下角分别由左下角、左上角复制得到。

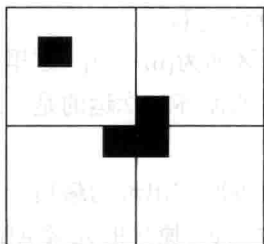


图 8-4 棋盘覆盖问题的递归解法

1	2	3	4	5	6	7	8
2	1	4	3	6	5	8	7
3	4	1	2	7	8	5	6
4	3	2	1	8	7	6	5
5	6	7	8	1	2	3	4
6	5	8	7	2	1	4	3
7	8	5	6	3	4	1	2
8	7	6	5	4	3	2	1

图 8-5 循环日程表问题 $k=3$ 时的解

巨人与鬼。在平面上有 n 个巨人和 n 个鬼，没有三者在同一条直线上。每个巨人需要选择一个不同的鬼，向其发送质子流消灭它。质子流由巨人发射，沿直线行进，遇到鬼后消失。由于质子流交叉是很危险的，所有质子流经过的线段不能有交点。请设计一种给巨人和鬼配对的方法。

【分析】

由于只需要一种配对方法，从直观上来说本题一定是有解的。由于每一个巨人和鬼都需要找一个目标，不妨先给“最特殊”的巨人或鬼寻找“搭档”。

考虑 y 坐标最小的点（即最低点）。如果有多个这样的点，考虑最左边的点（即其中最左边的点），则所有点的极角在范围 $[0, \pi)$ 内。不妨设它是一个巨人，然后把所有其他点按照极角从小到大的顺序排序后依次检查。

情况 1：第一个点是鬼，那么配对完成，剩下的巨人和鬼仍然是一样多，而且不会和这一条线段交叉，如图 8-6 (a) 所示。

情况 2：第一个点是巨人，那么继续检查，直到已检查的点中鬼和巨人一样多为止。找到了这个“鬼和巨人”配对区间后，只需要把此区间内的点配对，再把区域外的点配对即可，如图 8-6 (b) 所示。这个配对过程是递归的，好比棋盘覆盖中一样。会不会找不到这样的配对区间呢？不会的。因为检查完第一个点后鬼少一个，而检查完最后一个点时鬼多一个，而巨人和鬼的数量差每次只能改变 1，因此“从少到多”的过程中一定会有“一样多”的时候。